Lloyd k-means methodology, same run on one host

k = 2 · 31,702,685 rows · same initial centres · stop when the largest centre moves by at most 1e-4

Input matrix
6 standardised fields, all rows

→

Initial centres
the same k = 2 file for every engine

→

Stop rule
max shift ≤ 1e-4, at most 300 rounds

MapReduce on YARN — one round

The round is repeated. Measured runs stopped at round 86.

Map stage

Splitting
HDFS splits of the full file

→

Mapper
nearest of the 2 centres

→

Emit
cluster id, row, and 1

Reduce stage

Shuffle and sort
group by cluster id

→

Reducer
sum and count → new centre

Map-slot limit 4, 8, or 12. Every row is kept.
This is not a local top-N filter.

Spark local mode — one round

The same stop rule. Measured runs also stopped at round 86.

Spark session

→

Cached RDD
all 31.7M rows stay in memory

Driver
holds the 2 centres

→

local[N]
N = 4, 8, or 12 on this PC

→

Executors
N threads share the rows

One round on those threads

Broadcast
2 centres go out

→

Map
nearest centre for every row

→

Reduce
sums and counts by cluster

→

Collect
2 new centres

Driver and executors share goodpc.
No extra virtual machines, and no separate cluster.

Python, 1 thread

The same assign-and-update loop runs in one process. Numeric libraries are pinned to 1 thread.
Wall clock for Python, Spark, and MapReduce starts after the matrix is loaded. One host: goodpc, 6 physical cores / 12 threads.